Lab 6: Vending Machine FSM with Test Bench

CMPE 125 · Vivado 2025.2 · October 20, 2026

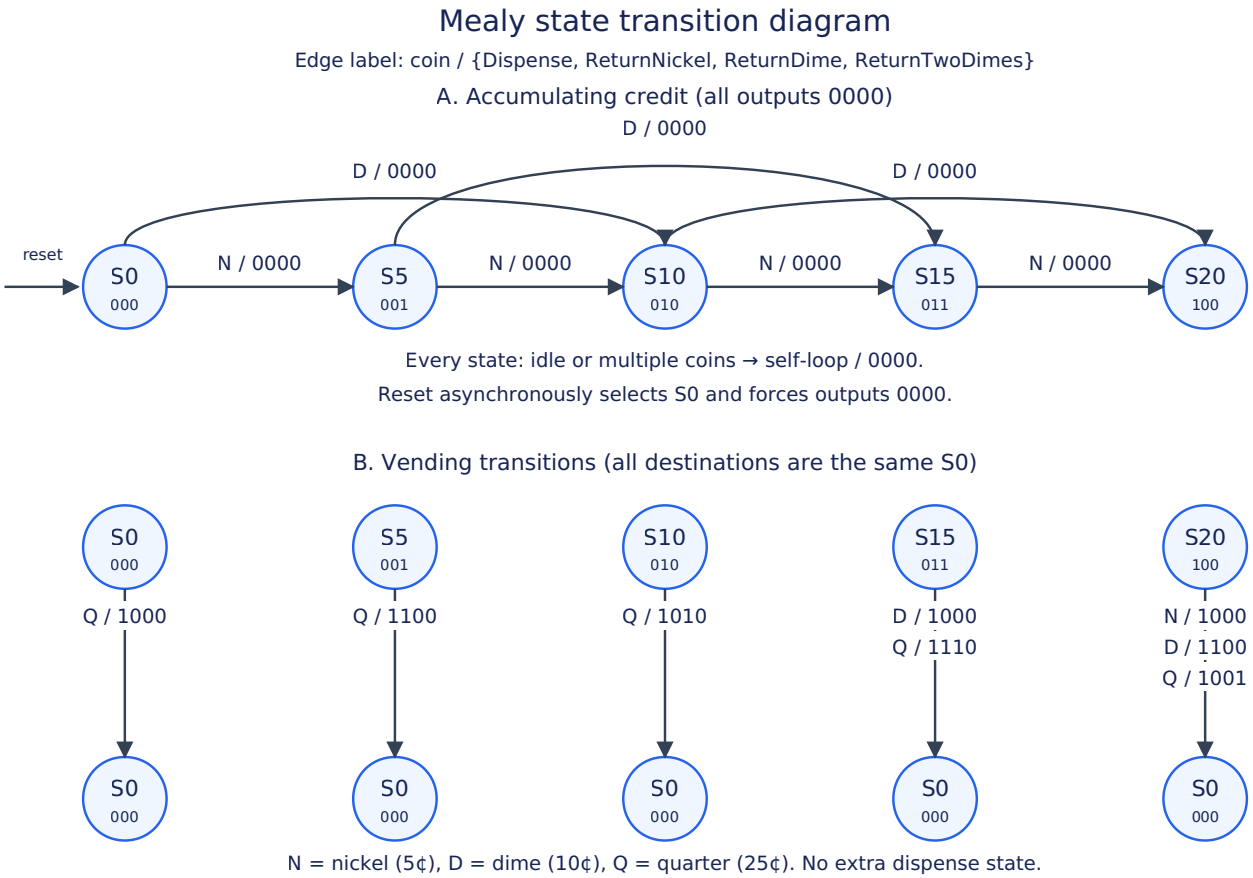
Objective and design choices

Design a controller that accepts nickels, dimes, and quarters for a 25-cent soda. When the current credit plus the inserted coin reaches or exceeds 25 cents, assert `Dispense`, return the excess, and prepare for the next purchase. The handout permits either a Mealy or Moore FSM. This design uses a Mealy FSM with five credit states and three state bits. Its outputs depend on both the stored credit and the current coin, allowing another purchase on the next clock without an extra dispense state.

`clk` samples on its rising edge. `reset` is active high and asynchronous, clears the credit to zero, and suppresses all outputs. A synchronous consumer samples the Mealy output with the coin at the rising edge; the outputs may change immediately after the credit changes. The testbench therefore checks outputs before the rising edge and checks the updated state afterward. Coins are driven at falling edges and cleared 1 ns after their sampling edges.

The assignment assumes exactly one coin per cycle. The implementation also defines idle (`Nickel,Dime,Quarter = 000`) and multiple-coin combinations: credit holds and outputs are zero. Unused state encodings recover to `S0` on the next rising edge and produce zero outputs before recovery. Reset clears credit without issuing change; refund/cancel behavior is outside the handout.

State transition diagram



The diagram is split into accumulation and vending views for readability. Repeated `S0` nodes denote the same zero-credit state. Together the views show every one-coin transition. The self-loop rule covers idle and all multiple-coin inputs; reset overrides every transition.

State encoding

State	Stored credit	x2 x1 x0
S0	0 cents	000
S5	5 cents	001
S10	10 cents	010
S15	15 cents	011
S20	20 cents	100
Unused	Invalid	101, 110, 111

Five states require `ceil(log2(5)) = 3` D flip-flops. Binary state encoding represents credit divided by five. The RTL's `fsm_encoding = "none"` attribute keeps the specified encoding during synthesis.

State transition and output tables

Let `Y = {Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes}`. A return value of `1110` means dispense a soda and return one nickel plus one dime (15 cents). `1001` returns two dimes (20 cents); `ReturnDime` stays zero in that case.

Symbolic transitions

Each cell is **next state / Y**. `N`, `D`, and `Q` indicate exactly one coin. `H` includes idle and invalid simultaneous-coin combinations.

Present state	N (5 cents)	D (10 cents)	Q (25 cents)	H
S0	S5 / 0000	S10 / 0000	S0 / 1000	S0 / 0000
S5	S10 / 0000	S15 / 0000	S0 / 1100	S5 / 0000
S10	S15 / 0000	S20 / 0000	S0 / 1010	S10 / 0000
S15	S20 / 0000	S0 / 1000	S0 / 1110	S15 / 0000
S20	S0 / 1000	S0 / 1100	S0 / 1001	S20 / 0000

Encoded transitions

Each cell is **D2 D1 D0 / Y**. Input columns use the raw order `Nickel,Dime,Quarter`; `H = {000,011,101,110,111}`.

x2 x1 x0	N = 100	D = 010	Q = 001	H
000	001 / 0000	010 / 0000	000 / 1000	000 / 0000
001	010 / 0000	011 / 0000	000 / 1100	001 / 0000
010	011 / 0000	100 / 0000	000 / 1010	010 / 0000
011	100 / 0000	000 / 1000	000 / 1110	011 / 0000
100	000 / 1000	000 / 1100	000 / 1001	100 / 0000
101, 110, 111	000 / 0000	000 / 0000	000 / 0000	000 / 0000

For `reset=1`, every state and input combination selects `000` and `Y=0000`. The asynchronous clear acts immediately, without waiting for a clock edge.

Next-state and output logic

Use `+` for OR, adjacency for AND, and `~` for NOT. Lowercase `n`, `d`, and `q` decode exactly one asserted coin input. Capital `N`, `D`, and `Q` here refer to the raw Nickel, Dime, and Quarter input pins.

```
n = N ~D ~Q
d = ~N D ~Q
q = ~N ~D Q
h = ~(n + d + q)

E0 = ~x2 ~x1 ~x0
E1 = ~x2 ~x1  x0
E2 = ~x2  x1 ~x0
E3 = ~x2  x1  x0
E4 =  x2 ~x1 ~x0
```

The state decoders `E0` through `E4` correspond to `S0` through `S20`. They all evaluate to zero for an unused encoding. For `reset=0`, the D inputs are obtained by OR-ing all transition terms whose destination has that bit set:

```
D2 = h E4 + n E3 + d E2
D1 = h(E2 + E3) + n(E1 + E2) + d(E0 + E1)
D0 = h(E1 + E3) + n(E0 + E2) + d E1
```

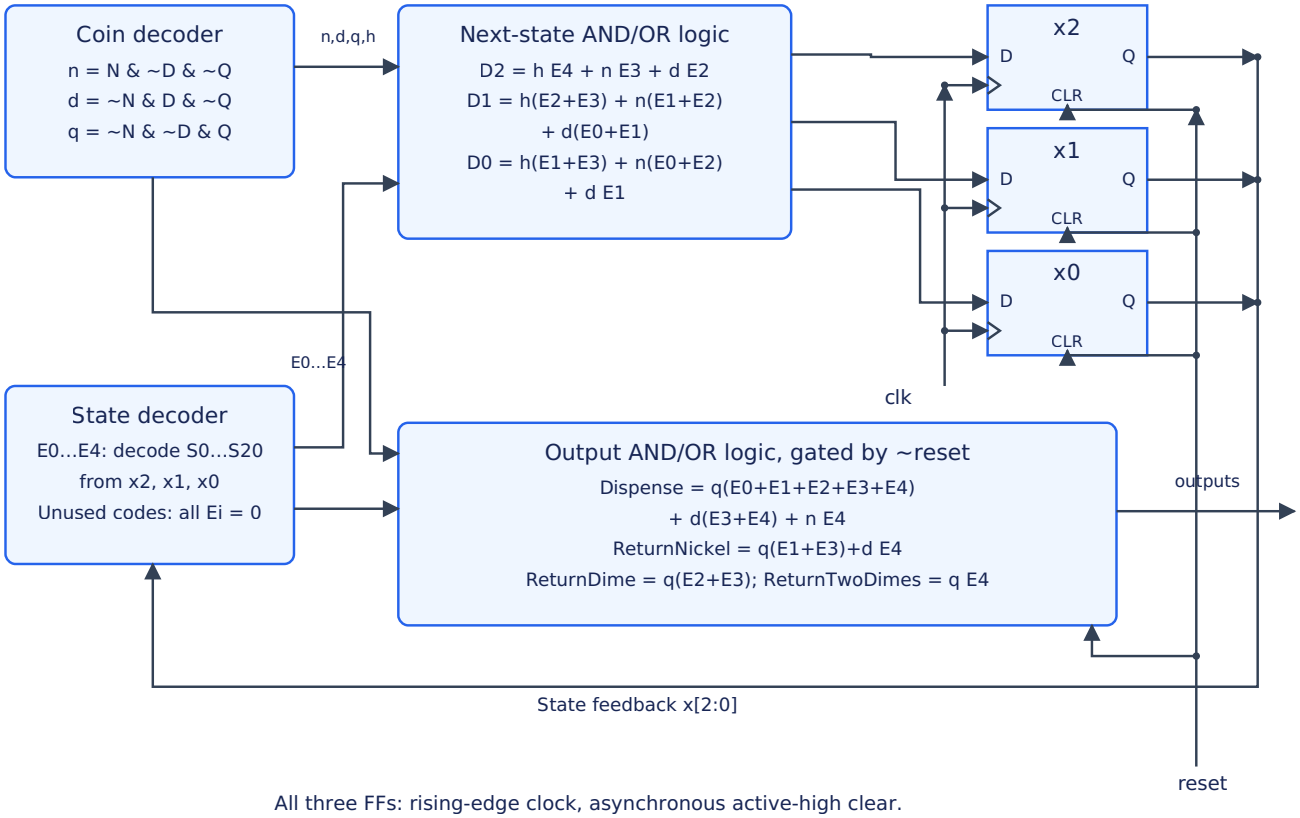
For example, `D2=1` only when holding at 20 cents, adding a nickel at 15 cents, or adding a dime at 10 cents. Every vending transition has `D2 D1 D0 = 000`. The resettable FFs implement `x_next = 000` while reset is asserted; the RTL also sets the combinational `next_state` to zero during reset.

```
Dispense = ~reset [q(E0+E1+E2+E3+E4) + d(E3+E4) + n E4]
ReturnNickel = ~reset [q(E1+E3) + d E4]
ReturnDime = ~reset [q(E2+E3)]
ReturnTwoDimes = ~reset [q E4]
```

The maximum stored credit is 20 cents. Adding the largest coin, a quarter, produces 45 cents, so the largest required change is 20 cents. The output combinations represent every possible change: 0, 5, 10, 15, and 20 cents.

Schematic and Verilog implementation

FSM schematic: combinational logic + three resettable D flip-flops



All three FFs: rising-edge clock, asynchronous active-high clear.

h = ~(n | d | q). Equations use + for OR and adjacency for AND.

The coin decoder and five state decoders feed the AND/OR networks shown in the equations. Three positive-edge D flip-flops store `x2,x1,x0`, share `clk`, and share asynchronous active-high clear. Their Q outputs feed back into the state decoder. Separate combinational output logic produces all four actuator signals and is gated by `~reset`.

The synthesizable source is vending_machine.v. It uses one sequential block with nonblocking assignments and one combinational block with blocking assignments and defaults. Every branch assigns or retains a defined next state and all outputs have defaults, avoiding inferred latches. The unused-state default recovers to zero credit. The design is a controller with synchronous coin sampling; a physical coin sensor would require its own synchronization and pulse generation.

Test vectors and testbench

The testbench extends Lab 5's Type 3 `$readmemb` approach. Inputs and expected results reside in the separate test_vector.tv file. Each 11-bit row has this format:

```
reset_NDQ_expectedOutputs_expectedState
  1    3         4              3        bits
```

For example, after one dime has established `S10`, the row `0_001_1010_000` applies a quarter, expects `Dispense=1` and `ReturnDime=1`, and expects `S0` after the rising edge. Underscores and `//` comments are accepted by `$readmemb`.

The testbench loads exactly 228 vectors. It drives each row at a falling edge, waits 1 ns, and compares all four Mealy outputs with case inequality (`!==`) so unknown outputs fail. It checks asynchronous reset at that same point, before the next clock. After the rising edge it waits 1 ns and checks the state. It then clears the coin, waits 1 ns, and verifies that no output remains asserted. An independent cents-based arithmetic model checks the expected outputs and next state in the vector file itself.

Directed transaction examples

Coin sequence	Total through vending coin	Change	Y on vending coin
Q	25 cents	0 cents	1000
N, Q	30 cents	5 cents	1100
D, Q	35 cents	10 cents	1010
N, D, Q	40 cents	15 cents	1110
D, D, Q	45 cents	20 cents	1001
D, D, D	30 cents	5 cents	1100
N, N, N, N, N	25 cents	0 cents	1000

The handout lists `Q+D`, six nickels, and `D+D+N+N` as possible overpayment examples. Under its immediate-vend rule, `Q` alone already buys a soda, the fifth nickel already buys a soda, and `D+D+N` already buys a soda. The remaining coin belongs to the next purchase. The vector file includes these sequences and checks those exact boundaries. Actual overpayment is covered by `D+Q`, `D+D+D`, and the other examples above.

Coverage and failure handling

After directed transactions, the file visits each of the five credit states and applies all eight raw coin input combinations: 40 state/input pairs. A coverage bitmap requires every pair to have been checked. A second bitmap requires all nonzero change amounts: 5, 10, 15, and 20 cents. Additional vectors assert reset from each credit state while a quarter is present, checking reset priority and suppression of dispense/change.

After the vector file, three fault-injection tests force the unused encodings `101`, `110`, and `111`, verify zero outputs even with a quarter, release the force, and verify recovery to `S0` at the next rising edge. Fault injection is simulation-only. Missing vectors, mismatched expectations, output/state errors, incomplete coverage, unsafe state recovery, and timeout all stop with `$fatal`. Purchases need no inserted recovery cycle; consecutive quarters verify consecutive vending cycles.

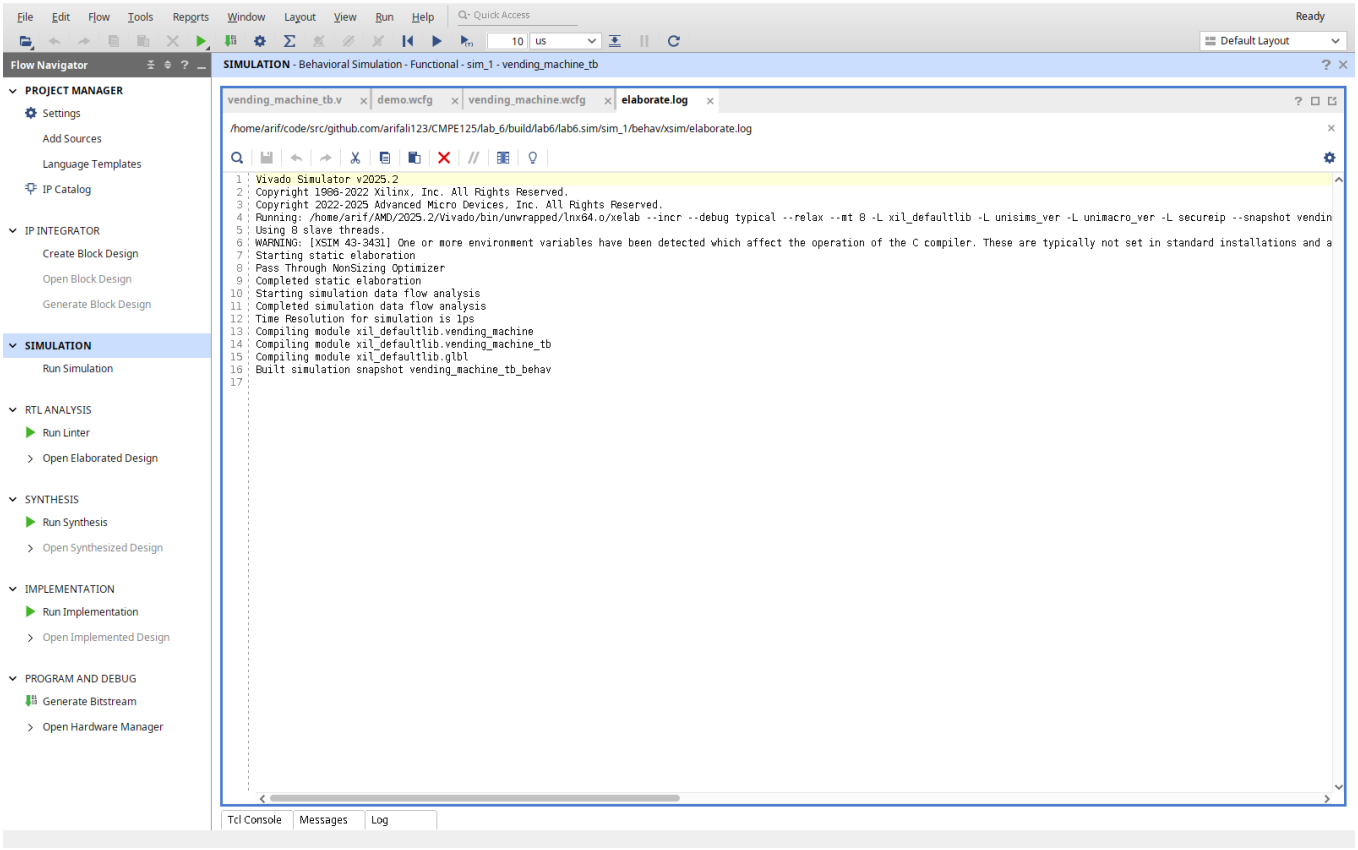
Compilation and simulation results

Verification was performed locally with **Vivado/XSim 2025.2** targeting `xc7a35tcpg236-1`. The design and testbench compiled and elaborated successfully. Synthesis also completed with zero warnings or errors, using three asynchronously cleared flip-flops, ten LUT sites, and zero latches. Simulation completed at **2317 ns**. The final transcript was:

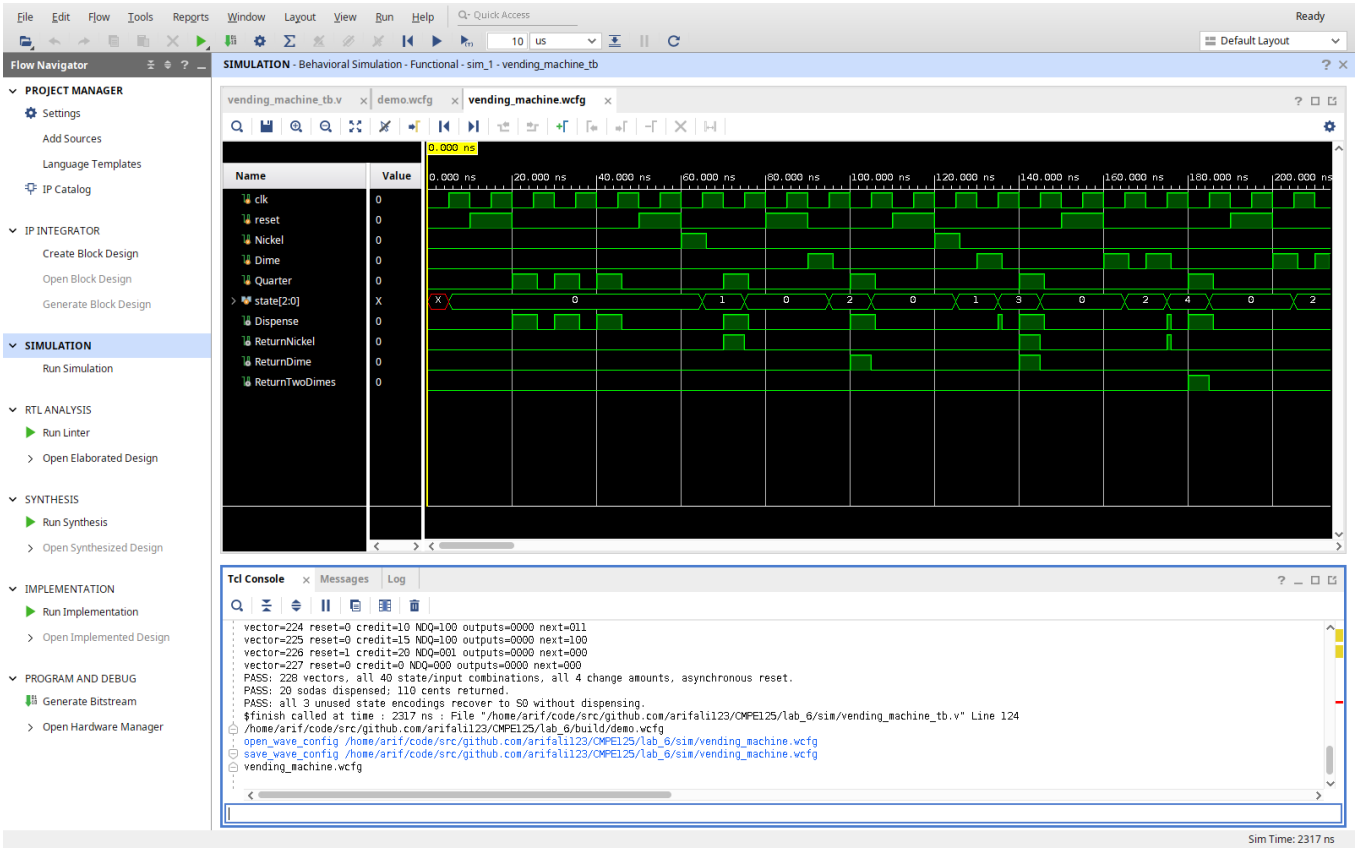
```
PASS: 228 vectors, all 40 state/input combinations, all 4 change amounts, asynchronous reset.
PASS: 20 sodas dispensed; 110 cents returned.
PASS: all 3 unused state encodings recover to S0 without dispensing.
```

The soda/change totals refer to the vector tests. Three unused-state checks follow them and are excluded from the purchase totals. XSim issued an environment warning about the host's `LIBRARY_PATH`; compilation, elaboration, and the self-checks completed successfully.

Compilation evidence



Waveform and self-checking simulation evidence



The directed tests at the beginning of the waveform show a quarter vending with no change, followed by return values of 5, 10, 15, and 20 cents. The state returns to `000` at each vending edge. Observe the output immediately before the rising edge because this is a Mealy controller.

To reproduce the verification from the repository root:

```
mkdir -p lab_6/build
(cd lab_6/build && vivado -mode batch -source ../scripts/verify.tcl)
```

To inspect interactively, run `./lab open lab_6` and select **Run Simulation > Run Behavioral Simulation**. The project marks `test_vector.tv` as a memory initialization file so Vivado copies it to the simulation working directory. The code and vectors remain portable when the repository moves.

Conclusion

The five-state controller dispenses at 25 cents, returns the correct change for every reachable excess amount, and immediately resets its credit for the next purchase. File-driven tests verified every state/input combination, all change outputs, asynchronous reset, consecutive purchases, and recovery from unused state encodings.

The provided handout references a separate Canvas report guideline which was not attached. This report includes the design artifacts, source files, vectors, and actual tool evidence requested in the handout; cover-page and submission formatting should be checked against that separate guideline.

Appendix A: synthesizable Verilog

```
`timescale 1ns / 1ps

// Mealy controller: sample coin inputs and outputs at the rising clock edge.
// 000 holds credit; multiple asserted coin inputs are ignored.
module vending_machine (
    input clk,
    input reset,
    input Nickel,
    input Dime,
    input Quarter,
    output reg Dispense,
    output reg ReturnNickel,
    output reg ReturnDime,
    output reg ReturnTwoDimes
);
    localparam S0  = 3'b000,
               S5  = 3'b001,
               S10 = 3'b010,
               S15 = 3'b011,
               S20 = 3'b100;

    // Preserve the report's binary encoding during synthesis.
    (* fsm_encoding = "none" *) reg [2:0] state;
    reg [2:0] next_state;

    always @(posedge clk or posedge reset) begin
        if (reset) state <= S0;
        else       state <= next_state;
    end

    always @* begin
        next_state = state;
        {Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes} = 4'b0000;
        case (state)
            S0: case ({Nickel, Dime, Quarter})
                3'b100: next_state = S5;
                3'b010: next_state = S10;
                3'b001: begin next_state = S0; Dispense = 1'b1; end
                default: ;
            endcase
            S5: case ({Nickel, Dime, Quarter})
                3'b100: next_state = S10;
                3'b010: next_state = S15;
                3'b001: begin
                    next_state = S0;
                    Dispense = 1'b1; ReturnNickel = 1'b1;
                end
                default: ;
            endcase
            S10: case ({Nickel, Dime, Quarter})
                3'b100: next_state = S15;
                3'b010: next_state = S20;
                3'b001: begin
                    next_state = S0;
                    Dispense = 1'b1; ReturnDime = 1'b1;
                end
                default: ;
            endcase
            S15: case ({Nickel, Dime, Quarter})
                3'b100: next_state = S20;
                3'b010: begin next_state = S0; Dispense = 1'b1; end
```

```
                3'b001: begin
                    next_state = S0;
                    Dispense = 1'b1; ReturnNickel = 1'b1; ReturnDime = 1'b1;
                end
                default: ;
            endcase
            S20: case ({Nickel, Dime, Quarter})
                3'b100: begin next_state = S0; Dispense = 1'b1; end
                3'b010: begin
                    next_state = S0;
                    Dispense = 1'b1; ReturnNickel = 1'b1;
                end
                3'b001: begin
                    next_state = S0;
                    Dispense = 1'b1; ReturnTwoDimes = 1'b1;
                end
                default: ;
            endcase
            default: next_state = S0;
        endcase
        if (reset) begin
            next_state = S0;
            {Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes} = 4'b0000;
        end
    end
endmodule
```

Appendix B: simulation testbench

```
`timescale 1ns / 1ps

// Lab 5 Type 3 style, extended to check stateful transactions.
module vending_machine_tb;
    parameter NUM_VECTORS = 228;
    reg clk = 0;
    reg reset = 0;
    reg Nickel = 0, Dime = 0, Quarter = 0;
    wire Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes;
    reg [10:0] testvectors [0:NUM_VECTORS-1];
    reg [3:0] expected_outputs;
    reg [2:0] expected_state;
    reg [7:0] coverage [0:4];
    integer vectornum, errors, i;
    integer credit, coin_value, total, change, reference_state;
    integer vends, returned;
    reg [3:0] reference_outputs;
    reg [3:0] change_coverage;

    vending_machine uut (
        .clk(clk), .reset(reset),
        .Nickel(Nickel), .Dime(Dime), .Quarter(Quarter),
        .Dispense(Dispense), .ReturnNickel(ReturnNickel),
        .ReturnDime(ReturnDime), .ReturnTwoDimes(ReturnTwoDimes)
    );
    always #5 clk = ~clk;

    initial begin
        $dumpfile("vending_machine.vcd");
        $dumpvars(0, vending_machine_tb);
        $readmemb("test_vector.tv", testvectors);
        errors = 0; credit = 0; vends = 0; returned = 0;
        change_coverage = 0;
        for (i = 0; i < 5; i = i + 1) coverage[i] = 0;

        for (vectornum = 0; vectornum < NUM_VECTORS; vectornum = vectornum + 1) begin
            // Drive halfway between sampling edges. Mealy outputs must be
            // checked BEFORE the edge that updates the credit register.
            @(negedge clk);
            {reset, Nickel, Dime, Quarter, expected_outputs, expected_state}
                = testvectors[vectornum];
            if ((^testvectors[vectornum]) === 1'bx)
                $fatal(1, "Missing/unknown vector %0d", vectornum);

            // Independent arithmetic model checks the vector file as well.
            reference_outputs = 0;
            reference_state = credit / 5;
            coin_value = 0;
            change = 0;
            if (reset) reference_state = 0;
            else begin
                coverage[credit / 5][{Nickel, Dime, Quarter}] = 1'b1;
                case ({Nickel, Dime, Quarter})
                    3'b100: coin_value = 5;
                    3'b010: coin_value = 10;
                    3'b001: coin_value = 25;
                    default: coin_value = 0;
                endcase
                total = credit + coin_value;
                if (total >= 25) begin
                    change = total - 25;
```

```
                    reference_outputs = {1'b1, (change % 10 == 5),
                                         (change == 10 || change == 15),
                                         (change == 20)};
                    reference_state = 0;
                    vends = vends + 1;
                    returned = returned + change;
                    if (change != 0) change_coverage[change / 5 - 1] = 1'b1;
                end else reference_state = total / 5;
            end
            if (expected_outputs !== reference_outputs || expected_state !== reference_state[2:0])
                $fatal(1, "Incorrect expectations in vector %0d", vectornum);
            #1;
            if ({Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes} !== expected_outputs) begin
                $display("ERROR vector %0d: credit=%0d NDQ=%b%b%b outputs=%b expected=%b",
                    vectornum, credit, Nickel, Dime, Quarter,
                    {Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes}, expected_outputs);
                errors = errors + 1;
            end
            if (reset && uut.state !== 3'b000)
                $fatal(1, "Asynchronous reset failed before clock at vector %0d", vectornum);
            $display("vector=%0d reset=%b credit=%0d NDQ=%b%b%b outputs=%b next=%b",
                vectornum, reset, credit, Nickel, Dime, Quarter, expected_outputs, expected_state);
            @(posedge clk); #1;
            if (uut.state !== expected_state) begin
                $display("ERROR vector %0d: state=%b expected=%b", vectornum, uut.state, expected_state);
                errors = errors + 1;
            end
            credit = reference_state * 5;
            // Deassert the sampled coin; an idle input must produce no output.
            {Nickel, Dime, Quarter} = 3'b000;
            #1;
            if ({Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes} !== 4'b0000)
                $fatal(1, "Output stuck after coin removal at vector %0d", vectornum);
        end
        for (i = 0; i < 5; i = i + 1)
            if (coverage[i] !== 8'hff)
                $fatal(1, "Missing state/input coverage: state=%0d mask=%b", i, coverage[i]);
        if (change_coverage !== 4'b1111)
            $fatal(1, "Missing change amounts: %b", change_coverage);
        // Exercise the unused encodings without adding nonphysical coin inputs.
        for (i = 5; i < 8; i = i + 1) begin
            @(negedge clk);
            Quarter = 1'b1;
            case (i)
                5: force uut.state = 3'b101;
                6: force uut.state = 3'b110;
                7: force uut.state = 3'b111;
            endcase
            #1;
            if (uut.next_state !== 3'b000 ||
                {Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes} !== 4'b0000)
                $fatal(1, "Unsafe unused state %0d", i);
            release uut.state;
            @(posedge clk); #1;
            if (uut.state !== 3'b000) $fatal(1, "Unused state did not recover");
            Quarter = 1'b0;
        end
        #1;
        if (errors != 0) $fatal(1, "FAIL: %0d errors", errors);
        $display("PASS: %0d vectors, all 40 state/input combinations, all 4 change amounts, asynchronous reset.",
NUM_VECTORS);
        $display("PASS: %0d sodas dispensed; %0d cents returned.", vends, returned);
        $display("PASS: all 3 unused state encodings recover to S0 without dispensing.");
        $finish;
    end
    initial begin
        #100000;
```

```
        $fatal(1, "Simulation timeout");
    end
endmodule
```

Appendix C: input and expected-output vectors

```
// Lab 6: {reset, Nickel, Dime, Quarter, Dispense, ReturnNickel,
//         ReturnDime, ReturnTwoDimes, expected_state[2:0]}
// Outputs are sampled BEFORE the rising edge; state is checked AFTER it.
// State encodes stored credit / 5. NDQ: N=100, D=010, Q=001.

// Quarter purchases, including consecutive sodas
1_000_0000_000
0_001_1000_000
0_001_1000_000
0_001_1000_000

// N + Q: 30 cents, return 5 cents
1_000_0000_000
0_100_0000_001
0_001_1100_000

// D + Q: 35 cents, return 10 cents
1_000_0000_000
0_010_0000_010
0_001_1010_000

// N + D + Q: 40 cents, return 15 cents
1_000_0000_000
0_100_0000_001
0_010_0000_011
0_001_1110_000

// D + D + Q: 45 cents, return two dimes
1_000_0000_000
0_010_0000_010
0_010_0000_100
0_001_1001_000

// D + D + D: 30 cents, return 5 cents
1_000_0000_000
0_010_0000_010
0_010_0000_100
0_010_1100_000

// Five nickels: exact payment
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100
0_100_1000_000

// Six nickels: vend on fifth, sixth starts next purchase
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100
0_100_1000_000
0_100_0000_001

// Q + D: vend on Q, D starts next purchase
1_000_0000_000
0_001_1000_000
0_010_0000_010
```

```
// D + D + N + N: vend on first N, second N starts next purchase
1_000_0000_000
0_010_0000_010
0_010_0000_100
0_100_1000_000
0_100_0000_001

// Exhaustive: credit=0, NDQ=000
1_000_0000_000
0_000_0000_000

// Exhaustive: credit=0, NDQ=001
1_000_0000_000
0_001_1000_000

// Exhaustive: credit=0, NDQ=010
1_000_0000_000
0_010_0000_010

// Exhaustive: credit=0, NDQ=011
1_000_0000_000
0_011_0000_000

// Exhaustive: credit=0, NDQ=100
1_000_0000_000
0_100_0000_001

// Exhaustive: credit=0, NDQ=101
1_000_0000_000
0_101_0000_000

// Exhaustive: credit=0, NDQ=110
1_000_0000_000
0_110_0000_000

// Exhaustive: credit=0, NDQ=111
1_000_0000_000
0_111_0000_000

// Exhaustive: credit=5, NDQ=000
1_000_0000_000
0_100_0000_001
0_000_0000_001

// Exhaustive: credit=5, NDQ=001
1_000_0000_000
0_100_0000_001
0_001_1100_000

// Exhaustive: credit=5, NDQ=010
1_000_0000_000
0_100_0000_001
0_010_0000_011

// Exhaustive: credit=5, NDQ=011
1_000_0000_000
0_100_0000_001
0_011_0000_001

// Exhaustive: credit=5, NDQ=100
1_000_0000_000
0_100_0000_001
0_100_0000_010

// Exhaustive: credit=5, NDQ=101
1_000_0000_000
0_100_0000_001
```

```
0_101_0000_001

// Exhaustive: credit=5, NDQ=110
1_000_0000_000
0_100_0000_001
0_110_0000_001

// Exhaustive: credit=5, NDQ=111
1_000_0000_000
0_100_0000_001
0_111_0000_001

// Exhaustive: credit=10, NDQ=000
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_000_0000_010

// Exhaustive: credit=10, NDQ=001
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_001_1010_000

// Exhaustive: credit=10, NDQ=010
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_010_0000_100

// Exhaustive: credit=10, NDQ=011
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_011_0000_010

// Exhaustive: credit=10, NDQ=100
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011

// Exhaustive: credit=10, NDQ=101
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_101_0000_010

// Exhaustive: credit=10, NDQ=110
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_110_0000_010

// Exhaustive: credit=10, NDQ=111
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_111_0000_010

// Exhaustive: credit=15, NDQ=000
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_000_0000_011
```

```
// Exhaustive: credit=15, NDQ=001
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_001_1110_000

// Exhaustive: credit=15, NDQ=010
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_010_1000_000

// Exhaustive: credit=15, NDQ=011
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_011_0000_011

// Exhaustive: credit=15, NDQ=100
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100

// Exhaustive: credit=15, NDQ=101
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_101_0000_011

// Exhaustive: credit=15, NDQ=110
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_110_0000_011

// Exhaustive: credit=15, NDQ=111
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_111_0000_011

// Exhaustive: credit=20, NDQ=000
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100
0_000_0000_100

// Exhaustive: credit=20, NDQ=001
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100
0_001_1001_000

// Exhaustive: credit=20, NDQ=010
1_000_0000_000
```

```
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100
0_010_1100_000

// Exhaustive: credit=20, NDQ=011
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100
0_011_0000_100

// Exhaustive: credit=20, NDQ=100
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100
0_100_1000_000

// Exhaustive: credit=20, NDQ=101
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100
0_101_0000_100

// Exhaustive: credit=20, NDQ=110
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100
0_110_0000_100

// Exhaustive: credit=20, NDQ=111
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100
0_111_0000_100

// Asynchronous reset from credit=0, with Quarter asserted
1_000_0000_000
1_001_0000_000
0_000_0000_000

// Asynchronous reset from credit=5, with Quarter asserted
1_000_0000_000
0_100_0000_001
1_001_0000_000
0_000_0000_000

// Asynchronous reset from credit=10, with Quarter asserted
1_000_0000_000
0_100_0000_001
0_100_0000_010
1_001_0000_000
0_000_0000_000

// Asynchronous reset from credit=15, with Quarter asserted
1_000_0000_000
0_100_0000_001
```

```
0_100_0000_010
0_100_0000_011
1_001_0000_000
0_000_0000_000

// Asynchronous reset from credit=20, with Quarter asserted
1_000_0000_000
0_100_0000_001
0_100_0000_010
0_100_0000_011
0_100_0000_100
1_001_0000_000
0_000_0000_000
```